

ScholarMate - AI Learning Platform

Comprehensive Technical Documentation & System Reference Guide

Project Name:	ScholarMate AI Learning Platform
Institution:	AANM & VVRSR Polytechnic College
Department:	Diploma in Computer Engineering & Information Technology
Project Type:	Final Year Major Project (2026)
Core Tech Stack:	Node.js, Express, SQLite (sql.js), Google Gemini 2.5 Flash, Tesseract OCR
Live Repository:	https://github.com/project202651/scholarmate

1. Executive Summary & Project Abstract

ScholarMate is an advanced, AI-powered web learning platform engineered to revolutionize how students study from textbooks, lecture slides, and scanned notes. By uploading PDF textbooks or image photos, ScholarMate automatically extracts document text using high-performance PDF parsing and Optical Character Recognition (OCR), then leverages **Google Gemini 2.5 Flash** AI to transform raw content into structured study assets: **Executive Notes**, **Interactive 3D Flashcards**, and **Multiple Choice Quizzes**.

The system addresses a critical challenge in modern education: the time-consuming manual effort required to read lengthy textbooks, summarize core concepts, format study cards, and construct self-assessment exams. ScholarMate automates this entire pipeline in seconds.

2. System Architecture & High-Level Pipeline

ScholarMate follows a modular 3-Tier Architecture comprising a client-side Single Page Application (SPA), a RESTful Express API gateway, and an AI/OCR processing engine backed by a WebAssembly SQLite database.

Layer	Component	Responsibility
Frontend	HTML5, CSS3, Vanilla JS	Renders SPA UI, 3D flip card animations, theme toggle, stats, & quiz runner.
API Server	Node.js + Express.js	Handles JWT auth, multer file uploads, route dispatches, and CORS security.
Text Extraction	pdf-parse & Tesseract.js	Extracts text from selectable PDFs and runs OCR on scanned image files.

AI Generation	Google Gemini 2.5 Flash	Synthesizes deep JSON notes, flashcards, & MCQs from document text.
Database	SQLite (sql.js WASM)	Stores users, upload library, notes, cards, and quiz scores without native C++ deps.

3. Core Modules & Key Features

A. Document Parsing & Dual OCR Fallback Engine

ScholarMate handles both digital PDFs and physical scanned textbook photos:

- **Digital PDFs:** Extracted using pdf-parse for maximum accuracy and speed.
- **Scanned Image PDFs & Photos:** Automatically triggers Tesseract.js OCR when selectable text is missing.
- **Full Document Window:** Preserves up to 500,000 characters (~125,000 words), ensuring complete textbook coverage without cutting off middle or ending chapters.

B. AI Notes Generation Engine

Transforms raw document text into structured, high-yield study notes containing:

- **Executive Summary:** 3 comprehensive paragraphs detailing core background & significance.
- **Key Takeaways:** 15-20 bullet points highlighting exact definitions, formulas, and theorems.
- **Exam Questions & Solutions:** Realistic conceptual questions paired with thorough answers.
- **One-Click Export:** Allows students to download notes as clean .txt files.

C. Interactive 3D Flip Flashcards Carousel

- **3D CSS Flip Effect:** Cards flip seamlessly with CSS preserve-3d perspective.
- **Interactive Controls:** Previous/Next buttons, Deck Shuffle (Fisher-Yates algorithm), and Card Counter.
- **Keyboard Shortcuts:** Press **Spacebar** to flip cards and **Left/Right Arrow keys** to navigate.

D. Interactive MCQ Quiz & Scoring Engine

- **10-Question Exams:** Generates multiple-choice questions covering all document chapters.
- **Real-Time Scoring:** Computes percentage score with instant answer key feedback.
- **AI Explanations:** Explains why the correct option is right for every question.

E. Smart Local Fallback Engine (Zero API Key Blockage)

- When GEMINI_API_KEY is configured in .env, ScholarMate uses **Google Gemini 2.5 Flash**.
- If no API key is set, the system automatically falls back to an intelligent local text analyzer that samples sentences evenly across all document sections, ensuring zero downtime or error popups.

4. Relational Database Schema (SQLite via sql.js WASM)

ScholarMate utilizes sql.js (WebAssembly SQLite) to provide synchronous database operations with zero native C++ compiler dependencies. Tables are auto-created and persisted to `scholarmate.db`:

```
-- Database Schema Overview (scholarmate.db)

CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE books (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  filename TEXT NOT NULL,
  original_name TEXT NOT NULL,
  file_type TEXT NOT NULL,
  extracted_text TEXT NOT NULL,
  page_count INTEGER DEFAULT 1,
  uploaded_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE notes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  book_id INTEGER NOT NULL,
  user_id INTEGER NOT NULL,
  summary TEXT NOT NULL,
  bullet_points TEXT NOT NULL,      -- Stored as JSON string
  important_questions TEXT NOT NULL, -- Stored as JSON string
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE flashcards (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  book_id INTEGER NOT NULL,
  user_id INTEGER NOT NULL,
  cards_json TEXT NOT NULL,        -- Array of {front, back} JSON
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE quizzes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  book_id INTEGER NOT NULL,
  user_id INTEGER NOT NULL,
  questions_json TEXT NOT NULL,    -- Array of MCQ questions JSON
  score INTEGER DEFAULT NULL,
  completed INTEGER DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

5. Local Execution & Cloud Deployment Guide

Local Setup Instructions:

```
# 1. Clone & Install
git clone https://github.com/project202651/scholarmate.git
cd scholarmate
npm install

# 2. Configure Environment (.env)
PORT=3000
GEMINI_API_KEY=your_gemini_api_key_here
JWT_SECRET=scholarmate_secret_key_2026

# 3. Start Application
npm start
# Server runs live on http://localhost:3000
```

Free Cloud Deployment (Render.com):

1. Sign up on **Render.com** and connect your GitHub repository.
2. Select **Web Service** with Runtime: Node, Build Command: npm install, Start Command: npm start.
3. Add Environment Variable GEMINI_API_KEY and click **Deploy**.